AI Usage Log- Sprint-2
Kavya Patel

Task ID: SCRUM-170: Fix Server Connection and Startup Crash

Purpose of AI Use: To debug a "Failed to fetch" frontend error and a terminal crash indicating missing environment variables.

Prompt: "I am getting an error while testing I am trying to log in but not working" (with screenshot) and "thi is the error i am getting. help me login to apply" (with terminal screenshot).

Response: AI identified that the Node server was crashing immediately on startup because the .env file was missing in the server/src directory, preventing the backend from listening to requests.

AI-Suggested Content: Instructions to create an .env file with the exact MONGO_URI, JWT_SECRET, and PORT variables.

Validation: Created the .env file, restarted the server, and verified that the backend successfully started on port 3000 and the frontend login fetch requests worked.

Decision: Accepted.

Reflection: AI was incredibly useful in diagnosing the exact cause of a silent frontend failure by analyzing the corresponding backend terminal logs and identifying missing configuration files.

Responsible Person: Kavya Patel

Task ID: SCRUM-105: Git History Cleanup and Merge Conflict Resolution

Purpose of AI Use: To remove old unrelated commits from a pull request, safely isolate Sprint 2 features, and resolve a Git cherry-pick merge conflict.

Prompt: "HOW DO I REMOVE EVERYTHING EXCEPT SCRUM 161 AND 162" and shared screenshots of a merge conflict and Vim editor terminal.

Response: AI provided step-by-step Git commands to reset the branch, cherry-pick the exact commit hashes, resolve the JobDetail.jsx conflict by replacing the file contents, and type :wq to save and exit the Vim editor.

AI-Suggested Content: Terminal commands including git checkout main, git branch -D, git cherry-pick, git push -f, and instructions for navigating Vim.

Validation: Ran the terminal commands, successfully cleared the pull request history on GitHub to only show the two relevant commits, and resolved the merge conflict locally.

Decision: Accepted.

Reflection: AI provided excellent guidance for advanced Git operations, effectively helping me clean up a messy branch history without losing my new work.

Responsible Person: Kavya Patel

Task ID: SCRUM-228: Fix GitHub Actions Pipeline ESLint Errors

Purpose of AI Use: To resolve strict linter errors that were causing the continuous integration (CI) pipeline to fail on GitHub.

Prompt: Shared screenshots of GitHub Actions failing with "'useAuth' is not defined" and "'err' is defined but never used".

Response: AI identified the exact lines in JobDetail.jsx causing the failures and provided the missing import statement and a console.error fix to satisfy the linter.

AI-Suggested Content: Adding import { useAuth } from '../context/AuthContext'; at the top of the file and adding console.error("Fetch error:", err); inside the catch blocks to utilize the variable.

Validation: Applied the code changes, pushed to GitHub, and verified the GitHub Actions workflow completed successfully with a green checkmark.

Decision: Accepted.

Reflection: AI helped decipher the automated pipeline logs and quickly resolve code quality warnings that were blocking the PR from being merged.

Responsible Person: Kavya Patel

Task ID: SCRUM-161 & SCRUM-162: Job Application Submission API and Frontend Flow

Purpose of AI Use: To implement the backend Express routes, database queries, and React frontend components for job application submissions.

Prompt: "help me finish scrum161 and scrum 162 of scrum 105. things that are assigned to me" and give me step by step things to do to finish scrum 161.[attached codes]

Response: AI provided the updated Mongoose schema as the previous one provided was wrong. That matches exact acceptance criteria, the backend POST route and database queries, and the complete React component (JobDetail.jsx) utilizing useState and fetch.

AI-Suggested Content: Complete code snippets for application.js model, applicationQueries.js, applicationRoutes.js, and JobDetail.jsx.

Validation: Tested the backend locally by starting the server and submitting an application through the frontend UI to ensure it was successfully saved to the database and conditionally rendered the success message.

Decision: Accepted and modified before use.

Reflection: AI helped bridge the gap between the frontend forms and backend queries, ensuring all acceptance criteria (like specific tracking statuses) were accurately met.

Task ID: SCRUM-142: Code Review Fixes & Pipeline Integration

Purpose of AI Use: To implement code review feedback (loading states, disabled buttons) and debug a GitHub Actions pipeline failure.

Prompt: "Could we add a loading state while fetching jobs? ... disable the Apply Now button... Web App Testing Pipeline workflow run has failed. How to fix it [attached pipeline error image]"

Response: AI provided the React state updates for loading/errors and then diagnosed the pipeline failure as a Node 20 deprecation and a React Fast Refresh ESLint error.

AI-Suggested Content: React code snippets for conditional rendering, the eslint-disable comment for AuthContext.jsx, and a YAML update to change the runner to Node 24.

Validation: Applied the React UI changes, updated the YAML configuration, pushed the new branch, and verified that the GitHub Actions pipeline passed successfully.

Decision: Accepted.

Reflection: AI efficiently resolved strict linting and CI/CD environment errors, ensuring the code met both the reviewer's UI requirements and the automated testing pipeline standards.

Responsible Person: Kavya Patel